

THE FREE VIBE CODING STACK

7 Free Tools That Make Vibe Coding 10x Better

From idea to design, build, test, debug and deploy: a practical workflow for shipping real software with an AI coding agent.

INTRODUCTION

Vibe coding needs more than an AI agent

Describing an app to an AI agent and watching it appear is exciting. But the first version is the easy part. The hard parts come after: the UI looks inconsistent, the data disappears on refresh, a login flow breaks, a real user hits a bug you never saw, and every deploy is a manual scramble.

None of these problems are about the AI being bad at writing code. They are about missing structure around the code. Professional developers solve them with a workflow: design, components, backend, tests, monitoring, deployment, automation. The good news is that every step has an excellent tool with a free tier.

The workflow at a glance

Stage	Tool	The question it answers
Design	Figma	What should this look like?
Build UI	shadcn/ui	How do I build it fast and consistently?
Backend	Supabase	Where does the data and login live?
Test	Playwright	Does it actually work for a real user?
Debug	Sentry	What broke in production, and for whom?
Deploy	Vercel	How do I get it online?
Automate	GitHub Actions	How do I stop doing this by hand?

A note on "free"

All seven tools offer a free tier or are open source, but limits and terms change. For example, Vercel's free Hobby plan is intended for personal, non-commercial projects, and Supabase free projects can pause after a period of inactivity. Check each tool's current pricing page before you launch something commercial.

Figma

Design before you code

What it is

Figma is a collaborative design tool that runs in the browser. You sketch screens, link them into clickable prototypes, and define reusable components such as buttons, cards and form fields.

Use it to

- Plan your screens and user flow before any code exists
- Create clickable prototypes to test an idea with real people
- Build a small set of reusable components and styles (colors, fonts, spacing)
- Give your coding agent a visual reference so the result matches your intent

How to use it with your AI coding agent

- Share a screenshot or export of your design and tell the agent to match it
- Write down your design tokens (colors, font sizes, spacing) and put them in your project's instructions file
- Design one screen at a time; build it; then move to the next
- If you use an agent that supports Figma integration, link the frame directly

Example prompt

Here is a screenshot of my dashboard design. Build this screen in React with Tailwind. Match spacing, colors and layout. Use placeholder data for now.

Watch out

You do not need a pixel-perfect design. A rough wireframe of 3 to 5 key screens already removes most of the guesswork that makes AI-generated UIs look generic.

shadcn/ui

Stop building UI from scratch

What it is

shadcn/ui is an open-source collection of accessible, well-designed React components built on Tailwind CSS and Radix primitives. Unlike a typical library, you copy the component code into your own project, so you own it and can change anything.

Use it to

- Buttons, inputs, forms and validation patterns
- Dialogs, dropdowns, tabs and toasts
- Data tables and dashboard layouts
- Sidebars and navigation

How to use it with your AI coding agent

- Tell the agent explicitly: use shadcn/ui components, do not invent custom ones
- Ask it to add components through the shadcn CLI so they match the official versions
- Keep one theme file so every new screen inherits the same look
- Because components live in your repo, the agent can read and adapt them easily

Example prompt

Set up shadcn/ui in this Next.js project. Then build a settings page using Card, Tabs, Form, Input and Button components. Reuse existing components instead of creating new ones.

Watch out

shadcn/ui is built for React and Tailwind. If your project uses another framework, check whether a community port exists before committing to it.

Supabase

Your backend in minutes

What it is

Supabase is an open-source backend platform built around PostgreSQL. It bundles a database, user authentication, file storage, auto-generated APIs and realtime subscriptions into one dashboard.

Use it to

- Store data in a real relational database
- Add email, password and social login
- Upload and serve files such as avatars or documents
- Push live updates to users (chat, notifications, collaborative views)

How to use it with your AI coding agent

- Create a project, then give the agent your project URL and public (anon) key through environment variables
- Ask the agent to write SQL migrations so your schema is versioned in the repo
- Have it design tables first, then build screens on top of them
- Ask it to enable Row Level Security and write policies for each table

Example prompt

Design a Postgres schema for a habit tracker with users, habits and daily check-ins. Write the SQL migration, enable Row Level Security so users can only see their own rows, then connect the frontend using the Supabase JS client.

Watch out

Never put your service_role key in frontend code or commit it to GitHub. It bypasses all security rules. And do not skip Row Level Security: without it, an exposed anon key can mean exposed data.

Playwright

Don't just build. Test.

What it is

Playwright is an open-source framework from Microsoft that controls real browsers (Chromium, Firefox and WebKit) through code. It clicks buttons, fills forms and checks that the right things appear, exactly like a user would.

Use it to

- Login and signup flows
- Forms and validation
- Checkout and payments (in test mode)
- Navigation and full user journeys

How to use it with your AI coding agent

- Ask the agent to write a test for every critical flow right after building it
- Run tests after each change so the agent catches regressions it just caused
- Paste failing test output back to the agent and ask it to fix the code, not weaken the test
- Use Playwright's trace viewer and screenshots on failure to see what went wrong

Example prompt

```
Write Playwright tests for signup, login and logout. Use data-testid selectors. Run them, and if any fail, fix the app code rather than changing the assertions.
```

Watch out

AI agents sometimes make a failing test pass by deleting or loosening it. Read what changed in the test file before you trust a green result.

Sentry

Find what broke in production

What it is

Sentry captures errors and performance data from your running app and groups them into issues. Each issue shows the stack trace, the browser or device, the user's steps leading up to it, and how many people were affected.

Use it to

- See errors and stack traces from real users
- Spot slow pages and slow API calls
- Prioritize bugs by how many users they affect
- Get alerts when a new issue appears after a deploy

How to use it with your AI coding agent

- Install the Sentry SDK for your framework and add your DSN as an environment variable
- When an issue appears, copy the stack trace and context into your agent and ask for a fix
- Ask the agent to add a regression test in Playwright for every bug you fix
- Tag releases so you can see which deploy introduced a problem

Example prompt

Here is a Sentry error: `TypeError: Cannot read properties of undefined (reading 'name')` in `ProfileCard.tsx` line 42. Find the root cause, fix it, and add a test that reproduces the bug.

Watch out

Do not send sensitive user data (passwords, tokens, personal details) to Sentry. Review its data-scrubbing settings, and be aware the free plan has limits on monthly events.

Vercel

Deploy without the headache

What it is

Vercel is a hosting platform optimized for modern web frameworks, especially Next.js. Connect a GitHub repository and every push becomes a live deployment, with no servers to manage.

Use it to

- Deploy automatically on every push to your main branch
- Get a unique preview URL for every branch or pull request
- Attach a custom domain with HTTPS
- Run backend code as serverless functions

How to use it with your AI coding agent

- Use preview deployments to review what the agent built before merging
- Set environment variables (Supabase keys, Sentry DSN) in the Vercel dashboard, never in code
- Ask the agent to fix build errors by pasting the deployment log
- Roll back instantly to a previous deployment if something breaks

Example prompt

Prepare this project for Vercel deployment. List every environment variable it needs, make sure the production build passes locally, and tell me what to set in the Vercel dashboard.

Watch out

The free Hobby plan has usage limits and is meant for non-commercial use. If you are building a business, plan for a paid plan or an alternative host.

GitHub Actions

Automate the boring stuff

What it is

GitHub Actions runs workflows automatically when something happens in your repository, such as a push or a pull request. A workflow is a YAML file in `.github/workflows` that lists the steps to run.

Use it to

- Run linting and type checks on every push
- Run your Playwright tests automatically
- Build the app and block merging if anything fails
- Trigger deployments or scheduled jobs

How to use it with your AI coding agent

- Ask the agent to write the workflow file; it is short and well-documented
- Turn on branch protection so code cannot merge unless the checks pass
- Store secrets in GitHub's encrypted secrets, never in the workflow file
- Keep the pipeline fast so you actually wait for it

Example prompt

```
Create a GitHub Actions workflow that runs on every pull request: install dependencies, run lint, run the type check, run Playwright tests, then build. Cache dependencies to keep it fast.
```

Watch out

Public repositories get free Actions minutes; private repositories have a monthly free allowance that can run out if your pipeline is slow or runs too often.

One idea, seven steps

Here is how the stack works on a real project: a simple habit-tracking app.

1	Figma	Sketch the dashboard, habit list and check-in screen. Note colors and fonts.
2	shadcn/ui	Tell the agent to build the screens from shadcn components, matching the sketch.
3	Supabase	Create tables for users, habits and check-ins. Add login. Turn on Row Level Security.
4	Playwright	Write tests for signup, adding a habit and checking it off.
5	GitHub Actions	Run those tests automatically on every pull request.
6	Vercel	Merge to main and it deploys. Review the preview link before merging.
7	Sentry	A real user hits a bug. Sentry shows the trace. The agent fixes it and adds a test.

Notice the loop at the end: Sentry finds a bug, the agent fixes it, Playwright locks the fix in with a test, Actions runs that test forever, and Vercel ships the patch. That loop is what separates a demo from software people can rely on.

Don't adopt everything on day one

If you are new, start small. A good first stack is the agent plus shadcn/ui plus Supabase plus Vercel: you can build and ship a real app with just those. Add Playwright and GitHub Actions when you have something worth protecting, and Sentry as soon as real users arrive. Figma earns its place the moment your UI starts to feel messy.

Habits that make AI coding reliable

- **Work in small steps.** One feature per prompt. Commit after each working step so you can always go back.
- **Give the agent a project brief.** Keep a short instructions file (stack, folder structure, design tokens, rules like "always use shadcn components").
- **Review what it writes.** Especially anything touching auth, payments, database policies or secrets.
- **Keep secrets out of code.** Use environment variables locally, in Vercel and in GitHub Secrets.
- **Let tests be the judge.** If a change breaks a test, the change is wrong until proven otherwise.
- **Paste real errors.** The exact error message, stack trace or log gets far better fixes than a description.

Pre-launch checklist

- [] Design reviewed for the main screens (Figma)
- [] UI uses a consistent component set (shadcn/ui)
- [] Database has Row Level Security enabled (Supabase)
- [] No secret keys in the repository
- [] Critical flows covered by tests (Playwright)
- [] Tests run on every pull request (GitHub Actions)
- [] Error monitoring is live (Sentry)
- [] Environment variables set in production (Vercel)
- [] Rollback plan known: previous deployment can be restored

The important part

Don't use AI as a replacement for your development workflow. Use AI inside a good development workflow. That is how you go from "AI wrote my app" to "I can actually ship production software with AI."